Second Year Scheme of Examinations For

1. COMPUTER SCIENCE AND ENGINEERING
2. COMPUTER SCIENCE AND ENGINEERING – ARTIFICIAL INTELLIGENCE
3. COMPUTER SCIENCE AND ENGINEERING – DATA SCIENCE

Third Semester					
Group	**Code**	**Paper**	**L**	**P**	**C**
Theory Papers					
PC	ICT-201T	Foundations of Computer Science	4	-	4
PC	ICT-203T	Database Management Systems	3	-	3
PC	ICT-205T	Object Oriented Programming using C++	3	-	3
PC	ICT-207T	Design and Analysis of Algorithms	3	-	3
MDC		Multidisciplinary Course (Any one to be taken by the students)			
	ICT-213	Digital System Design	3	-	3
	ICT-215	Microprocessor Architecture	3	-	3
		* Any other course / paper offered by any other school in the Dwarka campus of the University			
		One of the following papers shall be offered to the student of a section by the school	3	-	3
PCE	ICT-209T	Computational Methods			
PCE	ICT-211T	Computer Graphics			
Practical/ Viva Voce					
PC	ICT-203P	Database Management Systems Lab	-	2	1
PC	ICT-205P	Object Oriented Programming using C++	-	2	1
PC	ICT-207P	Design and Analysis of Algorithms Lab	-	2	1
		#The laboratory course offered shall correspond to the Theory paper offered from ICT 209T, ICT211T).	-	2	1
PCE	ICT-209P	Computational Methods Lab			
PCE	ICT-211P	Computer Graphics Lab			
Total					**23**

Paper Code(s): ICT-201T	**L : 4**
Paper: Foundations of Computer Science	**C :4**
Prerequisites: None	
Marking Scheme: 1. Teachers Continuous Evaluation: 40 marks 2. Term-End Semester Examinations: 60 Marks	
Instructions for paper setter:	
1. There should be 9 questions in the term end examinations question paper. 2. The first (1st) question should be compulsory and cover the entire syllabus. This question should be objective, single line answers or short answer type question of total 12 marks. 3. Apart from question 1 which is compulsory, rest of the paper shall consist of 4 units as per the syllabus. Every unit shall have two questions covering the corresponding unit of the syllabus. However, the student shall be asked to attempt only one of the two questions in the unit. Individual questions may contain upto 5 sub-parts / sub-questions. Each Unit shall have a marks weightage of 12. 4. The questions are to be framed keeping in view the learning outcomes of the course / paper. The standard / level of the questions to be asked should be at the level of the prescribed textbook. 5. The requirement of (scientific) calculators / log-tables / data – tables may be specified if required.	
Course Outcomes (CO):	
CO1	Apply propositional and predicate logic, proof techniques, induction principles, and mathematical reasoning to formulate and solve problems arising in Computer Science.
CO2	Analyze sets, functions, relations, elementary number theory, Boolean algebra, and algebraic structures for modeling computational systems and solving problems in algorithms and cryptography.
CO3	Solve combinatorial problems and recurrence relations, including divide-and-conquer recurrences using the Master Theorem, and apply these techniques in the mathematical analysis of algorithms.
CO4	Apply graph theory, tree structures, and discrete mathematical models to the design and analysis of algorithms, computing systems, and applications in theoretical computer science.

Course Outcomes (CO) to Programme Outcomes (PO) Mapping (scale 1: low, 2: Medium, 3: High)

CO/PO	*PO01*	*PO02*	*PO03*	*PO04*	*PO05*	*PO06*	*PO07*	*PO08*	*PO09*	*PO10*	*PO11*	*PO12*
CO1	3	3	2	1	1	-	-	-	2	1	1	3
CO2	3	3	2	1	1	-	-	-	2	1	1	3
CO3	3	3	3	1	1	-	-	-	2	1	1	3
CO4	3	3	3	1	1	-	-	-	2	1	1	3

Unit I

Propositional logic, Logical connectives, Truth tables, Logical equivalence, Tautologies and contradictions, Normal forms (CNF and DNF), Predicate logic, Quantifiers, Rules of inference, Mathematical proofs (direct proof, proof by contraposition, proof by contradiction, proof by cases), Mathematical induction, Strong induction, Structural induction, Recursive definitions, Recursive algorithms, Principle of Inclusion and Exclusion, Pigeonhole Principle and applications, Mathematical reasoning in Computer Science.

Unit II

Sets and set operations, Cartesian products, Power sets, Countable and uncountable sets.
Functions, Injective, Surjective and Bijective functions, Composition and inverse of functions.
Relations and their properties, Equivalence relations, Partitions, Partial order relations, Posets, Hasse diagrams, Lattices.
Elementary Number Theory: Divisibility, Prime numbers, Fundamental Theorem of Arithmetic (statement and applications), Greatest Common Divisor (GCD), Least Common Multiple (LCM), Euclidean Algorithm, Extended Euclidean Algorithm, Linear Diophantine equations, Congruences, Modular arithmetic, Residue classes, Modular inverses, Chinese Remainder Theorem (statement, proof and applications), Fermat's Little Theorem (statement, proof and applications), Euler's Totient Function, Euler's Theorem, Introduction to RSA cryptosystem.
Boolean algebra, Boolean functions, canonical forms, minimization of Boolean expressions, Karnaugh maps, Quine-McCluskey minimization algorithm.
Introduction to Algebraic Structures: Semigroups, Monoids, Groups, Rings and Fields with applications in computing.

Unit III

Fundamental principles of counting, Permutations and combinations, Binomial and multinomial coefficients, Binomial theorem, Inclusion–Exclusion applications, Generating functions (introductory concepts).
Recurrence relations: Recursive sequences, Linear recurrence relations with constant coefficients, Homogeneous and non-homogeneous recurrence relations, Characteristic equation method, Repeated roots, Particular solutions, Initial conditions, Matrix method for solving recurrence relations, Fibonacci sequence and related recurrences, Generating-function approach (introductory concepts).
Divide-and-conquer recurrence relations, Recursion tree method, Master Theorem (statement and complete proof), Applications of recurrence relations in the analysis of recursive algorithms, Introduction to asymptotic growth of recursive functions.

Unit IV

Graphs: Basic terminology, Directed and undirected graphs, Weighted graphs, Graph representations, Graph

isomorphism, Degree sequences, Bipartite graphs, Connected graphs, Eulerian and Hamiltonian graphs, Graph traversals (Breadth First Search and Depth First Search), Shortest path concepts, Spanning trees, Minimum spanning trees (Prim's and Kruskal's algorithms), Graph colouring, Planar graphs, Matchings (introductory concepts), Hall's Marriage Theorem (statement and applications).
Trees: Rooted trees, Binary trees, Binary Search Trees (introductory concepts), Tree traversals, Expression trees, Huffman trees.

Textbooks:

[1] K. H. Rosen, Discrete Mathematics and Its Applications, 8th ed. New York, NY, USA: McGraw-Hill Education, 2019.
[2] J. P. Tremblay and R. Manohar, Discrete Mathematical Structures with Applications to Computer Science, McGraw Hill, Latest Edition

References:

[1] R. P. Grimaldi, Discrete and Combinatorial Mathematics: An Applied Introduction, 5th ed. Boston, MA, USA: Pearson, 2004.
[2] C. L. Liu and D. P. Mohapatra, Elements of Discrete Mathematics, 4th ed. New York, NY, USA: McGraw-Hill Education, 2017.
[3] N. Deo, Graph Theory with Applications to Engineering and Computer Science. New Delhi, India: Prentice Hall of India, Reprint Edition.
[4] V. Shoup, A Computational Introduction to Number Theory and Algebra, 2nd ed. Cambridge, U.K.: Cambridge University Press, 2009.
[5] R. L. Graham, D. E. Knuth, and O. Patashnik, Concrete Mathematics: A Foundation for Computer Science, 2nd ed. Boston, MA, USA: Addison-Wesley, 1994.

Paper Code(s): ICT-203T	**L : 3**
Paper: Database Management Systems	**C : 3**
Prerequisites: None	

Marking Scheme:
1. Teachers Continuous Evaluation: 40 marks
2. Term-End Semester Examinations: 60 Marks

Instructions for paper setter:
1. There should be 9 questions in the term end examinations question paper.
2. The first (1st) question should be compulsory and cover the entire syllabus. This question should be objective, single line answers or short answer type question of total 12 marks.
3. Apart from question 1 which is compulsory, rest of the paper shall consist of 4 units as per the syllabus. Every unit shall have two questions covering the corresponding unit of the syllabus. However, the student shall be asked to attempt only one of the two questions in the unit. Individual questions may contain upto 5 sub-parts / sub-questions. Each Unit shall have a marks weightage of 12.
4. The questions are to be framed keeping in view the learning outcomes of the course / paper. The standard / level of the questions to be asked should be at the level of the prescribed textbook.
5. The requirement of (scientific) calculators / log-tables / data – tables may be specified if required.

Course Outcomes (CO):

CO1	Design conceptual and logical database models using ER/EER modeling techniques and map them to relational database schemas.
CO2	Develop relational databases using SQL, including queries, views, stored procedures, functions, triggers, and integrity constraints.
CO3	Apply normalization techniques and transaction management principles to design efficient, reliable, and consistent relational database systems.
CO4	Analyze indexing, query optimization, concurrency control, recovery mechanisms, distributed databases, NoSQL databases, and emerging database technologies for modern computing applications.

Course Outcomes (CO) to Programme Outcomes (PO) Mapping (scale 1: low, 2: Medium, 3: High)

CO/PO	*PO01*	*PO02*	*PO03*	*PO04*	*PO05*	*PO06*	*PO07*	*PO08*	*PO09*	*PO10*	*PO11*	*PO12*
CO1	3	3	2	1	2	-	-	-	2	2	1	3
CO2	3	3	2	1	2	-	-	-	2	2	1	3
CO3	3	3	3	2	2	-	-	-	2	2	1	3
CO4	3	3	3	3	2	-	-	-	2	2	1	3

Unit I

Introduction to database systems, Characteristics of the database approach, Evolution of database systems, Database users and administrators, Applications of database systems, Advantages and limitations of DBMS, Database architecture, Three-schema architecture, Data independence, Database languages (DDL, DML, DCL, TCL), Database system environment.

Data models, Entity-Relationship (ER) model, Enhanced Entity-Relationship (EER) model, Entities, Attributes, Relationships, Keys, Cardinality constraints, Participation constraints, Weak entities, Specialization, Generalization, Aggregation, Conceptual database design, Mapping ER/EER diagrams to relational schemas.

Unit II

Relational model, Domains, Relations, Tuples, Attributes, Super keys, Candidate keys, Primary keys, Alternate keys, Foreign keys, Integrity constraints (Entity integrity and Referential integrity), Relational algebra, Relational calculus (introductory concepts).

Structured Query Language (SQL): Data Definition Language (DDL), Data Manipulation Language (DML), Data Control Language (DCL), Transaction Control Language (TCL).

SQL programming: Database creation, Table creation, Constraints, INSERT, UPDATE, DELETE, SELECT, Ordering, Aggregate functions, GROUP BY, HAVING, Joins (Inner, Outer, Self and Cross joins), Nested queries, Set operations (UNION, INTERSECT, EXCEPT), Views, Sequences, Indexes, Stored procedures, Functions, Triggers, Introduction to embedded SQL.

Unit III

Functional dependencies, Attribute closure, Canonical cover, Lossless decomposition, Dependency preservation.

Normalization: First Normal Form (1NF), Second Normal Form (2NF), Third Normal Form (3NF), Boyce-Codd Normal Form (BCNF), Introduction to Fourth Normal Form (4NF) and Fifth Normal Form (5NF).

Transaction processing, ACID properties, Transaction states, Schedules, Serial and concurrent schedules, Conflict serializability, View serializability (introductory concepts), Concurrency control, Lock-based protocols, Two-Phase Locking (2PL), Timestamp ordering protocol, Deadlocks, Deadlock prevention and detection, Database recovery, Log-based recovery, Checkpointing.

Unit IV

Physical database design, File organization, Indexing techniques, B-Trees, B+ Trees, Hash indexing, Query processing, Query optimization (introductory concepts).

Distributed databases (overview), Client-server database architecture, Object-oriented databases (introductory concepts), Object-relational databases, NoSQL databases (Key-value, Document, Column-family and Graph

databases), CAP theorem (introductory concepts), Database security and authorization, Backup and recovery, Introduction to data warehousing, OLTP and OLAP systems, Applications of database systems.

Textbooks:

1. Abraham Silberschatz, Henry F. Korth, and S. Sudarshan, *Database System Concepts*, 7th Edition, McGraw-Hill Education, 2020.

References:

1. Ramez Elmasri and Shamkant B. Navathe, *Fundamentals of Database Systems*, 7th Edition, Pearson, 2017.
2. Raghu Ramakrishnan and Johannes Gehrke, *Database Management Systems*, 3rd Edition, McGraw-Hill Education, 2002.
3. C. J. Date, *An Introduction to Database Systems*, 8th Edition, Addison-Wesley, 2006.
4. Thomas Connolly and Carolyn Begg, *Database Systems: A Practical Approach to Design, Implementation, and Management*, 6th Edition, Pearson, 2015.
5. Carlos Coronel and Steven Morris, *Database Systems: Design, Implementation, and Management*, 13th Edition, Cengage Learning, 2019.

Paper Code(s): ICT-203P	**P : 2**
Paper: Database Management System Lab	**C : 1**
Prerequisites: None	
Marking Scheme: 1. Teachers Continuous Evaluation: 40 marks 2. Term-End Semester Examinations: 60 Marks	
Instructions: **3.** The course objectives and course outcomes are identical to that of ICT-203T as this is the practical component of the corresponding theory paper. **4.** The practical list shall be notified by the teacher in the first week of the class commencement. **5.** There should be at least 10 practical exercises	

Paper Code(s): ICT-205T	**L : 3**
Paper: Object Oriented Programming using C++	**C : 3**
Prerequisites: None	

Marking Scheme:
1. Teachers Continuous Evaluation: 40 marks
2. Term-End Semester Examinations: 60 Marks

Instructions for paper setter:
1. There should be 9 questions in the term end examinations question paper.
2. The first (1st) question should be compulsory and cover the entire syllabus. This question should be objective, single line answers or short answer type question of total 12 marks.
3. Apart from question 1 which is compulsory, rest of the paper shall consist of 4 units as per the syllabus. Every unit shall have two questions covering the corresponding unit of the syllabus. However, the student shall be asked to attempt only one of the two questions in the unit. Individual questions may contain upto 5 sub-parts / sub-questions. Each Unit shall have a marks weightage of 12.
4. The questions are to be framed keeping in view the learning outcomes of the course / paper. The standard / level of the questions to be asked should be at the level of the prescribed textbook.
5. The requirement of (scientific) calculators / log-tables / data – tables may be specified if required.

Course Outcomes (CO):

CO1	Design and implement object-oriented programs using classes, objects, constructors, inheritance, encapsulation, and polymorphism in Modern C++.
CO2	Develop reusable software components using advanced C++ language features, operator overloading, templates, dynamic memory management, and resource management techniques.
CO3	Apply generic programming concepts, exception handling, the Standard Template Library, lambda expressions, and modern programming practices to solve engineering problems efficiently.
CO4	Design maintainable object-oriented software using UML, SOLID principles, and fundamental design patterns while applying Modern C++ in real-world software development.

Course Outcomes (CO) to Programme Outcomes (PO) Mapping (scale 1: low, 2: Medium, 3: High)

CO/PO	*PO01*	*PO02*	*PO03*	*PO04*	*PO05*	*PO06*	*PO07*	*PO08*	*PO09*	*PO10*	*PO11*	*PO12*
CO1	3	3	2	2	1	-	-	-	1	-	1	3
CO2	3	3	2	2	1	-	-	-	1	-	1	3
CO3	3	3	3	2	1	-	-	-	1	-	1	3
CO4	3	3	3	2	1	-	-	-	2	1	2	3

Unit I

Evolution of C++, Features of Modern C++ (C++11/C++14/C++17/C++20 overview), Structure of a C++ program, Tokens, Identifiers, Keywords, Variables, Constants, References, Type deduction using auto, Scope resolution operator, Operators and expressions, Input/output streams, Namespaces.

Object-oriented programming concepts: Objects, Classes, Abstraction, Encapsulation, Data hiding, Modularity, Inheritance, Polymorphism, Dynamic binding, Message passing.

Classes and objects, Access specifiers, Constructors (default, parameterized, copy), Destructor, Inline functions, Friend functions and friend classes, Static data members and member functions, Constant member functions, Objects as function arguments and return values, Delegating constructors, Defaulted and deleted functions.

Unit II

Operator overloading, Unary and binary operator overloading, Stream insertion and extraction operators.

Inheritance: Single, Multiple, Multilevel, Hierarchical and Hybrid inheritance, Access control, Constructors and destructors in inheritance, Virtual base classes.

Compile-time polymorphism, Function overloading, Function overriding, Virtual functions, Pure virtual functions, Abstract classes, Dynamic binding, Virtual destructors, Object slicing.

Dynamic memory management, new and delete, Copy constructor, Assignment operator, Rule of Three, Introduction to Rule of Five, Move constructor and move assignment (concepts), Resource Acquisition Is Initialization (RAII), Smart pointers (unique_ptr, shared_ptr, weak_ptr).

Unit III

Function templates, Class templates, Template specialization, Generic programming concepts.

Exception handling: Exception hierarchy, try, throw, catch, Multiple catch blocks, Stack unwinding, User-defined exceptions, noexcept, Assertions.

Lambda expressions, Function objects (functors), Iterators.

Standard Template Library (STL): Sequence containers (vector, deque, list), Associative containers (set, multiset, map, multimap), Unordered containers (unordered_set, unordered_map), Container adapters (stack, queue, priority_queue), STL algorithms (sort, find, binary_search, transform, count, copy, remove, etc.), Introduction to ranges (overview), Practical applications of STL.

Unit IV

File handling using stream classes, Text and binary files, Command-line arguments.

Introduction to Unified Modeling Language (UML): Class diagrams, Object diagrams, Sequence diagrams.

Software design principles: Coupling and cohesion, SOLID principles (overview), Interface-based programming, Generic programming.

Introduction to Design Patterns: Design pattern concepts, Creational, Structural and Behavioural patterns, Detailed study of **Singleton**, **Factory Method**, **Strategy**, and **Observer** patterns, Applications of object-oriented programming and Modern C

Textbooks:

[1] B. Stroustrup, *Programming: Principles and Practice Using C++*, 3rd ed. Boston, MA, USA: Addison-Wesley Professional, 2024.
[2] S. B. Lippman, J. Lajoie, and B. E. Moo, *C++ Primer*, 5th ed. Boston, MA, USA: Addison-Wesley Professional, 2012.
[3] N. M. Josuttis, *The C++ Standard Library: A Tutorial and Reference*, 2nd ed. Boston, MA, USA: Addison-Wesley Professional, 2012.
[4] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Boston, MA, USA: Addison-Wesley Professional, 1994.

References:

[1] B. Stroustrup, *The C++ Programming Language*, 4th ed. Boston, MA, USA: Addison-Wesley Professional, 2013.
[2] S. Meyers, *Effective Modern C++: 42 Specific Ways to Improve Your Use of C++11 and C++14*. Sebastopol, CA, USA: O'Reilly Media, 2014.
[3] A. Alexandrescu, *Modern C++ Design: Generic Programming and Design Patterns Applied*. Boston, MA, USA: Addison-Wesley Professional, 2001.
[4] R. C. Martin, *Clean Code: A Handbook of Agile Software Craftsmanship*. Upper Saddle River, NJ, USA: Pearson, 2008.
[5] M. Reddy, *API Design for C++*. Burlington, MA, USA: Morgan Kaufmann, 2011.
[6] H. Sutter and A. Alexandrescu, *C++ Coding Standards: 101 Rules, Guidelines, and Best Practices*. Boston, MA, USA: Addison-Wesley Professional, 2004.
[7] A. Williams, *C++ Concurrency in Action*, 2nd ed. Shelter Island, NY, USA: Manning Publications, 2019.
[8] J. Lakos, *Large-Scale C++ Software Design*. Boston, MA, USA: Addison-Wesley Professional, 1996.
[9] J. Lakos, *Large-Scale C++*, Vol. 1: *Process and Architecture*. Boston, MA, USA: Addison-Wesley Professional, 2019.
[10] K. Henney, *97 Things Every C++ Programmer Should Know*. Sebastopol, CA, USA: O'Reilly Media, 2020.
[11] R. C. Martin, *Agile Software Development: Principles, Patterns, and Practices*. Upper Saddle River, NJ, USA: Prentice Hall, Latest Edition.
[12] M. Fowler, *Refactoring: Improving the Design of Existing Code*, 2nd ed. Boston, MA, USA: Addison-Wesley Professional, 2018.

Paper Code(s): ICT-205P	**P : 2**
Paper: Object Oriented Programming using C++ Lab	**C : 1**
Prerequisites: None	
Marking Scheme: 1. Teachers Continuous Evaluation: 40 marks 2. Term-End Semester Examinations: 60 Marks	
Instructions: **1.** The course objectives and course outcomes are identical to that of ICT-205T as this is the practical component of the corresponding theory paper. **2.** The practical list shall be notified by the teacher in the first week of the class commencement. **3.** There should be at least 10 practical exercises	

Paper Code(s): ICT-207T	**L : 3**
Paper: Design and Analysis of Algorithms	**C : 3**
Prerequisites: None	
Marking Scheme: 1. Teachers Continuous Evaluation: 40 marks 2. Term-End Semester Examinations: 60 Marks	
Instructions for paper setter:	
1. There should be 9 questions in the term end examinations question paper. 2. The first (1st) question should be compulsory and cover the entire syllabus. This question should be objective, single line answers or short answer type question of total 12 marks. 3. Apart from question 1 which is compulsory, rest of the paper shall consist of 4 units as per the syllabus. Every unit shall have two questions covering the corresponding unit of the syllabus. However, the student shall be asked to attempt only one of the two questions in the unit. Individual questions may contain upto 5 sub-parts / sub-questions. Each Unit shall have a marks weightage of 12. 4. The questions are to be framed keeping in view the learning outcomes of the course / paper. The standard / level of the questions to be asked should be at the level of the prescribed textbook. 5. The requirement of (scientific) calculators / log-tables / data – tables may be specified if required.	
Course Outcomes (CO):	
CO1	Analyze the correctness, efficiency and computational complexity of algorithms using formal mathematical techniques
CO2	Design efficient solutions using divide-and-conquer and greedy paradigms and justify their correctness.
CO3	Develop optimization algorithms using dynamic programming and evaluate their computational trade-offs.
CO4	Classify computational problems using complexity theory and apply advanced algorithmic techniques including backtracking, branch-and-bound, approximation and randomized algorithms

Course Outcomes (CO) to Programme Outcomes (PO) Mapping (scale 1: low, 2: Medium, 3: High)

CO/PO	*PO01*	*PO02*	*PO03*	*PO04*	*PO05*	*PO06*	*PO07*	*PO08*	*PO09*	*PO10*	*PO11*	*PO12*
CO1	3	3	2	3	1	-	-	-	1	1	1	3
CO2	3	3	2	2	2	-	-	-	1	1	1	3
CO3	3	3	3	3	2	-	-	-	1	1	1	3
CO4	3	3	3	3	2	-	-	-	1	1	1	3

Unit I

Introduction to algorithm design and analysis, characteristics of algorithms, algorithm specification and pseudocode, computational models and performance measures, analysis of algorithms, best-case, average-case and worst-case analysis, asymptotic notations, growth of functions, logarithmic, polynomial and exponential growth rates, summation techniques for algorithm analysis.lower bounds, decision-tree model, comparison-based sorting lower bounds, Loop invariants concept, initialization, maintenance termination and proofs of correctness.Divide-and-conquer paradigm, merge sort, quick sort, deterministic and randomized quicksort, selection problem, median-of-medians algorithm, Strassen matrix multiplication, recurrence relations arising from these algorithms and their solutions using substitution method, iteration method, recursion-tree method and Master theorem.Non-Comparison based average case linear time soring algorithms: counting sort, radix sort and bucket sort.

Unit II

Dynamic programming methodology, overlapping subproblems, optimal substructure, principle of optimality, memoization and tabulation, dynamic programming versus divide-and-conquer, rod cutting problem, matrix chain multiplication, optimal binary search tree, longest common subsequence problem.

Greedy method, characteristics of greedy algorithms, greedy-choice property and optimal substructure, exchange arguments, comparison of Greedy v/s Dynamic approach using Knapsack problem and activity selection problem as case study. Huffman coding algorithm.

Unit III

Graph Algorithms : Analysis and correctness of Prim's and Kruskal's algorithm for finding minimum cost spanning trees, Dijkastra's and Bellman Ford algorithm for single source shortest path. Floyd Warshall algorithm for all pair shortest path.

String Matching algorithms: The Rabin Karp algorithm, sting matching using finite automata, The Knuth Morris Pratt algorithm.

Unit IV

Overview of Backtracking and Branch-and-Bound techniques for solving problems like N-Queens, graph coloring, Hamiltonian cycle, travelling salesman. Definition and concept of P, NP, NP-hard and NP-complete class of problems, Polynomial time verification and reduction, 3-CNF satisfiability problem, Cook-Levin theorem (statement and significance), NP-completeness proofs of some problems like Clique, Vertex Cover, Subset-Sum. Approximation algorithms for problems like Vertex Cover, The travelling salesman problem in Planar Graph with triangular inequality, the set covering problem.

Textbooks

1. Introduction to Algorithms, Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest and Clifford Stein, MIT Press, Latest Edition.
2. Algorithm Design, Jon Kleinberg and Éva Tardos, Pearson Education, Latest Edition.

References

1. A. V. Aho, J. E. Hopcroft and J. D. Ullman, The Design and Analysis of Computer Algorithms, Addison-Wesley, 1974.
2. E. Horowitz, S. Sahni and S. Rajasekaran, Fundamentals of Computer Algorithms, Universities Press, Latest Edition.
3. S. S. Skiena, The Algorithm Design Manual, Springer, Latest Edition.
4. R. Sedgewick and K. Wayne, Algorithms, Addison-Wesley, Latest Edition.
5. S. Dasgupta, C. H. Papadimitriou and U. V. Vazirani, Algorithms, McGraw-Hill Education, Latest Edition.
6. D. Harel and Y. Feldman, Algorithmics: The Spirit of Computing, Adisson-Wesley, Latest Edition.

Paper Code(s): ICT-207P	**P : 2**
Paper: Object Design and Analysis of Algorithms **Lab**	**C : 1**
Prerequisites: None	
Marking Scheme: 1. Teachers Continuous Evaluation: 40 marks 2. Term-End Semester Examinations: 60 Marks	
Instructions: **1.** The course objectives and course outcomes are identical to that of ICT-207T as this is the practical component of the corresponding theory paper. **2.** The practical list shall be notified by the teacher in the first week of the class commencement. **3.** There should be at least 10 practical exercises	

Paper Code(s): ICT-215	**L : 3**
Paper: Microprocessor Architecture	**C : 3**
Prerequisites: None	
Marking Scheme: 1. Teachers Continuous Evaluation: 40 marks 2. Term-End Semester Examinations: 60 Marks	
Instructions for paper setter:	
1. There should be 9 questions in the term end examinations question paper. 2. The first (1st) question should be compulsory and cover the entire syllabus. This question should be objective, single line answers or short answer type question of total 12 marks. 3. Apart from question 1 which is compulsory, rest of the paper shall consist of 4 units as per the syllabus. Every unit shall have two questions covering the corresponding unit of the syllabus. However, the student shall be asked to attempt only one of the two questions in the unit. Individual questions may contain upto 5 sub-parts / sub-questions. Each Unit shall have a marks weightage of 12. 4. The questions are to be framed keeping in view the learning outcomes of the course / paper. The standard / level of the questions to be asked should be at the level of the prescribed textbook. 5. The requirement of (scientific) calculators / log-tables / data – tables may be specified if required.	

Course Outcomes (CO):

CO1	Explain the architecture, instruction set organization, and operation of modern microprocessors.
CO2	Develop assembly language programs using processor instructions, addressing modes, procedures, and system resources.
CO3	Analyze processor datapaths, pipelining, memory hierarchy, and performance enhancement techniques.
CO4	Evaluate modern processor architectures and their applications in embedded, parallel, and intelligent computing systems.

Course Outcomes (CO) to Programme Outcomes (PO) Mapping (scale 1: low, 2: Medium, 3: High)

CO/PO	*PO01*	*PO02*	*PO03*	*PO04*	*PO05*	*PO06*	*PO07*	*PO08*	*PO09*	*PO10*	*PO11*	*PO12*
CO1	3	3	2	1	1	-	-	-	2	1	1	3
CO2	3	3	2	1	1	-	-	-	2	1	1	3
CO3	3	3	3	1	1	-	-	-	2	1	1	3
CO4	3	3	3	1	1	-	-	-	2	1	1	3

Unit I

Evolution of microprocessors, Von Neumann and Harvard architectures, CISC and RISC architectures, Processor organization, Register organization, Arithmetic Logic Unit (ALU), Control Unit, Instruction execution cycle.
Instruction Set Architecture (ISA), Instruction formats, Addressing modes, Data representation, Arithmetic, Logical, Branch, Shift, Compare, Stack and Control instructions.
Processor status flags, Exceptions and interrupts, Privileged and user modes, Overview of x86, ARM and RISC-V processor architectures.

Unit II

Assembly language fundamentals, Assemblers, Linkers and Loaders, Program structure, Labels, Directives, Constants and Variables.
Arithmetic and logical programming, Conditional and unconditional branching, Looping constructs, Procedures and subroutines, Stack organization, Parameter passing, Recursion (introductory concepts), String manipulation, Modular programming.
Program development tools, Debugging techniques, Program optimization and performance considerations.

Unit III

Register Transfer Level (RTL), Datapath organization, Hardwired and microprogrammed control.
Instruction pipelining, Pipeline hazards, Operand forwarding, Branch prediction (introductory concepts), Instruction-Level Parallelism (ILP), Superscalar processors (overview).
Memory hierarchy, Cache organization, Cache mapping techniques, Virtual memory (introductory concepts), Performance evaluation using CPU execution time, CPI and speedup.

Unit IV

Multicore processors, Symmetric multiprocessing, Shared memory systems, SIMD and vector processors, Graphics Processing Units (GPUs) (overview).
Embedded processors, ARM Cortex architecture (introductory concepts), RISC-V architecture (introductory concepts), Low-power processor design.
Processor communication interfaces, Memory interfaces, Peripheral interfaces (overview), Applications of microprocessors.

Textbooks

[1] B. Brey, The Intel Microprocessors: Architecture, Programming, and Interfacing, 9th ed. Boston, MA, USA: Pearson, 2021.

References

[2] D. A. Patterson and J. L. Hennessy, Computer Organization and Design: The Hardware/Software Interface, 6th ed. Cambridge, MA, USA: Morgan Kaufmann, 2021.
[3] J. L. Hennessy and D. A. Patterson, Computer Architecture: A Quantitative Approach, 6th ed. Cambridge, MA, USA: Morgan Kaufmann, 2019.
[4] A. K. Ray and K. M. Bhurchandi, Advanced Microprocessors and Peripherals: Architecture, Programming and Interfacing. New Delhi, India: McGraw-Hill Education, 2013.
[5] M. A. Mazidi, J. G. Mazidi, and R. D. McKinlay, The 8051 Microcontroller and Embedded Systems, 2nd ed. Hoboken, NJ, USA: Pearson, 2006.
[6] J. Yiu, The Definitive Guide to ARM Cortex-M3 and Cortex-M4 Processors, 3rd ed. Oxford, U.K.: Newnes, 2014.

Paper Code(s): ICT211T/ICT220T	**L : 3**
Paper: Computer Graphics	**C : 3**
Prerequisites: None	

Marking Scheme:
1. Teachers Continuous Evaluation: 40 marks
2. Term-End Semester Examinations: 60 Marks

Instructions for paper setter:
1. There should be 9 questions in the term end examinations question paper.
2. The first (1st) question should be compulsory and cover the entire syllabus. This question should be objective, single line answers or short answer type question of total 12 marks.
3. Apart from question 1 which is compulsory, rest of the paper shall consist of 4 units as per the syllabus. Every unit shall have two questions covering the corresponding unit of the syllabus. However, the student shall be asked to attempt only one of the two questions in the unit. Individual questions may contain upto 5 sub-parts / sub-questions. Each Unit shall have a marks weightage of 12.
4. The questions are to be framed keeping in view the learning outcomes of the course / paper. The standard / level of the questions to be asked should be at the level of the prescribed textbook.
5. The requirement of (scientific) calculators / log-tables / data – tables may be specified if required.

Course Outcomes (CO):

CO1	Apply fundamental computer graphics algorithms for scan conversion, two-dimensional transformations, viewing and clipping operations.
CO2	Develop and analyze three-dimensional graphical models using transformation, projection and geometric representation techniques.
CO3	Apply visible surface detection, illumination and shading algorithms for realistic rendering of graphical scenes.
CO4	Explain rendering, animation and modern graphics pipeline concepts and evaluate their applications in engineering and interactive visualization systems.

Course Outcomes (CO) to Programme Outcomes (PO) Mapping (scale 1: low, 2: Medium, 3: High)

CO/PO	*PO01*	*PO02*	*PO03*	*PO04*	*PO05*	*PO06*	*PO07*	*PO08*	*PO09*	*PO10*	*PO11*	*PO12*
CO1	3	3	2	2	2	-	-	-	-	-	-	1
CO2	3	3	2	2	2	-	-	-	-	1	-	2
CO3	3	3	3	2	2	-	-	-	-	1	-	2
CO4	3	3	3	2	2	-	-	-	1	1	1	3

Unit I

Introduction to computer graphics, applications of computer graphics, graphics systems, components of interactive graphics systems, graphics input and output devices, raster scan and random scan display systems, frame buffer, pixels and resolution, colour representation and graphics standards. Scan conversion algorithms: point plotting, Digital Differential Analyzer (DDA) algorithm, Bresenham's line drawing algorithm, midpoint circle algorithm, Bresenham's circle algorithm and midpoint ellipse algorithm. Anti-aliasing concepts. Two-dimensional geometric transformations: translation, scaling, rotation, reflection and shearing, homogeneous coordinate representation, composite transformations, transformations about arbitrary points. Two-dimensional viewing pipeline, window and viewport transformation, clipping concepts, point clipping, line clipping algorithms including Cohen–Sutherland and Liang–Barsky algorithms, polygon clipping using Sutherland–Hodgman algorithm.

Unit II

Three-dimensional coordinate systems, homogeneous coordinates and representation of three-dimensional objects. Three-dimensional transformations: translation, scaling, rotation, reflection, shearing, composite transformations, rotation about coordinate axes and arbitrary axes. Three-dimensional viewing pipeline, viewing coordinate systems, parallel projections including orthographic, axonometric and oblique projections, perspective projections, viewing transformations and viewport mapping. Geometric modeling: parametric representation of curves and surfaces, polynomial curves, Hermite curves, Bézier curves and their properties, B-spline curves and their properties. Surface representation using polygon meshes, Bézier surfaces and B-spline surfaces. Introduction to solid modeling concepts including boundary representation (B-rep) and constructive solid geometry (CSG).

Unit III

Visible surface determination, classification of hidden surface removal algorithms, object-space and image-space approaches, back-face detection, depth-buffer (Z-buffer) algorithm, scan-line algorithm, Painter's algorithm and Binary Space Partitioning (BSP) technique. Illumination models: light sources, ambient, diffuse and specular reflection models, Phong illumination model, surface properties and normal vectors. Shading techniques: flat shading, Gouraud shading and Phong shading, comparison of shading methods. Colour models: RGB, CMY, CMYK, HSV and HSL colour models and their applications.

Unit IV

Rendering concepts, texture mapping, texture representation and mapping techniques, shadow generation, reflection, transparency and refraction concepts. Introduction to ray tracing: ray generation, ray-object intersection, recursive ray tracing concepts and limitations. Computer animation principles, key-frame

animation, interpolation techniques, transformation-based animation, tweening and morphing. Introduction to graphics programming concepts, graphics primitives, coordinate systems, overview of graphics libraries and modern graphics pipelines including vertex processing, rasterization and fragment processing. Applications of computer graphics.

Textbooks

[1] D. Hearn and M. P. Baker, *Computer Graphics with C*, 2nd ed. Upper Saddle River, NJ, USA: Prentice Hall, 1997.
[2] D. Hearn, M. P. Baker, and W. Carithers, *Computer Graphics with OpenGL*, 4th ed. Boston, MA, USA: Pearson Education, 2014.

References

[1] J. D. Foley, A. van Dam, S. K. Feiner, and J. F. Hughes, *Computer Graphics: Principles and Practice*, 3rd ed. Boston, MA, USA: Addison-Wesley Professional, 2014.
[2] D. F. Rogers, *Procedural Elements for Computer Graphics*, 2nd ed. New York, NY, USA: McGraw-Hill, 1998.
[3] F. S. Hill Jr., *Computer Graphics Using OpenGL*, 3rd ed. Upper Saddle River, NJ, USA: Pearson Education, 2007.
[4] P. Shirley and S. Marschner, *Fundamentals of Computer Graphics*, 5th ed. Boca Raton, FL, USA: CRC Press, 2021.
[5] T. Akenine-Möller, E. Haines, and N. Hoffman, *Real-Time Rendering*, 4th ed. Boca Raton, FL, USA: CRC Press, 2018.
[6] J. Vince, *Mathematics for Computer Graphics*, 2nd ed. London, U.K.: Springer, 2017.

Paper Code(s): ICT211P/ICT220P	**P : 2**
Paper: Computational Methods Lab	**C : 1**
Prerequisites: None	
Marking Scheme: 1. Teachers Continuous Evaluation: 40 marks 2. Term-End Semester Examinations: 60 Marks	
Instructions: **1.** The course objectives and course outcomes are identical to that of ICT211T/ICT220T as this is the practical component of the corresponding theory paper. **2.** The practical list shall be notified by the teacher in the first week of the class commencement. **3.** There should be at least 10 practical exercises	